



École Marocaine des Sciences de l'Ingénieur de Tanger

Département Ingénierie Informatique et Réseaux

Programmation Python et Frameworks — 2025/2026

MINI-PROJET PYTHON

Projet n°5 : Gestion des présences aux séances de cours

PORTAL

Application Web de Gestion des Présences

v2.4.0 — Django 6.0.4 • React 19.2.5 • DRF 3.17.1

Réalisé par

BAKKALI Owais

AHRRAR Amjad

HABZ Amine

DOUKHANA Bachar

MESBAHI Bilal

Encadrante : Pr. Abroun Soundouss

Jury : Pr. Abroun Soundouss

Remerciements

Nous tenons à exprimer notre profonde gratitude envers notre encadrante, **Pr. Abroun Soundouss**, pour son accompagnement bienveillant, sa disponibilité et les précieux conseils prodigués tout au long de ce projet. Ses orientations techniques et méthodologiques ont été déterminantes pour la qualité du travail accompli.

Nous remercions l'École Marocaine des Sciences de l'Ingénieur de Tanger pour l'excellente formation dispensée et les ressources pédagogiques mises à notre disposition. Nos remerciements vont également à l'ensemble des enseignants du département Informatique, ainsi qu'à nos familles pour leur soutien constant.

Table des matières

Remerciements	i
Liste des figures	iv
1 Introduction générale	1
1.1 Contexte du projet	1
1.2 Objectifs du projet	1
1.3 Présentation de PORTAL	1
1.4 Plan du rapport	2
2 Architecture du système	3
2.1 Architecture globale — Découplée REST	3
2.2 Structure des dossiers du projet	3
2.3 Flux de données — Authentification et routing	3
3 Tables de la base de données	5
3.1 Application <code>accounts</code> — Utilisateurs et profils	5
3.2 Application <code>portal</code> — Entités académiques	6
3.3 Historique des migrations	8
4 Diagrammes de cas d'utilisation	9
4.1 Vue globale du système	9
4.2 Cas d'utilisation détaillé : Saisie des présences	9
4.3 Cas d'utilisation détaillé : Workflow justification d'absence	10
5 Diagramme de classes	11
6 Diagrammes de séquence	12
6.1 Séquence 1 : Connexion utilisateur	12
6.2 Séquence 2 : Saisie des présences (Enseignant)	13
6.3 Séquence 3 : Saisie groupée des notes	14
7 Relations entre les entités	15
7.1 Carte des relations	15
7.2 Description détaillée des relations	16
7.3 Sérialeurs imbriqués — Relations exposées à l'API	16
8 Bibliothèques utilisées et justification	17
8.1 Stack back-end (Python/Django)	17
8.2 Stack front-end (JavaScript/React)	18
8.3 Dépendances de développement	18
9 Captures d'écran	19
9.1 Authentification	19
9.2 Tableau de bord Administrateur	19
9.3 Tableau de bord Enseignant	21
9.4 Tableau de bord Étudiant	21

Table des figures

2.1	Architecture découplée REST de l'application PORTAL	3
4.1	Diagramme de cas d'utilisation global — PORTAL	9
5.1	Diagramme de classes simplifié — modèles principaux de PORTAL	11
6.1	Diagramme de séquence — Connexion utilisateur	12
6.2	Diagramme de séquence — Saisie des présences par un enseignant	13
6.3	Diagramme de séquence — Saisie groupée des notes par un enseignant	14
7.1	Carte complète des relations entre entités — PORTAL	15
9.1	Page de connexion (Login.jsx)	19
9.2	Vue d'ensemble Administrateur — statistiques (utilisateurs, documents en attente, absences)	19
9.3	Gestion des utilisateurs et affectation académique (AdminDashboard.jsx — renderUsers())	20
9.4	Emploi du temps et validations (AdminDashboard.jsx)	20
9.5	Dashboard et création de séance (TeacherDashboard.jsx)	21
9.6	Présences et notes (TeacherDashboard.jsx — renderAttendance(), renderGrades())	21
9.7	Emploi du temps et présences (StudentDashboard.jsx — renderCalendar(), renderAttendance())	21
9.8	Notes et absences (StudentDashboard.jsx — renderGrades(), renderAbsences())	22
9.9	Notifications et paramètres (StudentDashboard.jsx + TopBar.jsx)	22

Chapitre 1

Introduction générale

1.1 Contexte du projet

Dans les établissements d'enseignement supérieur, le suivi de la présence des étudiants aux séances de cours est une tâche critique. Lorsqu'elle est réalisée manuellement — feuilles papier ou fichiers Excel —, cette gestion devient rapidement difficile à organiser, sujette aux erreurs et inexploitable pour des analyses statistiques.

Dans le cadre du module *Programmation Python et Frameworks* (3^{ème} année IIR, EMSI Tanger), notre groupe a choisi de réaliser le **Projet n°5** : « Gestion des présences aux séances de cours ». Notre réalisation, baptisée **PORTAL**, va bien au-delà du périmètre minimal imposé par le cahier des charges pour constituer un écosystème académique complet.

1.2 Objectifs du projet

Objectifs pédagogiques (CDC) : mettre en pratique les concepts de programmation Python et développement web en réalisant une application fonctionnelle avec Django, appliquer les bonnes pratiques de développement logiciel, et travailler en équipe de 4 à 5 étudiants.

Objectifs techniques atteints :

- Comprendre et implémenter l'architecture d'une application web découplée (REST)
- Développer une application dynamique multi-rôles avec Python/Django et React
- Manipuler une base de données relationnelle SQLite via l'ORM Django
- Implémenter les opérations CRUD complètes pour toutes les entités
- Concevoir des interfaces modernes et réactives
- Structurer un projet logiciel en équipe avec Git

1.3 Présentation de PORTAL

PORTAL (v2.4.0) est une plateforme web complète dont le cœur est la **gestion des présences aux séances de cours**. Elle fournit trois interfaces dédiées — Administrateur, Enseignant, Étudiant — communiquant via une API REST sécurisée.

Fonctionnalité	Description
Gestion des présences	Création de séances, saisie présence/absence par étudiant, sauvegarde groupée
Gestion des utilisateurs	Étudiants, enseignants, admins avec profils complets et photo
Gestion des modules/-cours	Catalogue de cours, affectation aux enseignants avec classes
Notes et bulletins	Saisie groupée, recalcul automatique moyenne, relevé multi-années
Emploi du temps	Grille 5 jours par classe, éditeur admin, vue enseignant/étudiant
Absences et justifications	Workflow soumission → validation/rejet
Notifications	Broadcast admin/enseignant vers étudiants
Calendrier académique	Événements Cours/TD/Examen ciblés par filière
Documents administratifs	Attestation de scolarité / de réussite

Table 1.1. Fonctionnalités principales de PORTAL

1.4 Plan du rapport

Ce rapport est structuré selon les grandes phases de conception d'un projet logiciel : après cette introduction, le chapitre 2 présente l'architecture globale, le chapitre 3 décrit les tables de la base de données, le chapitre 4 expose les diagrammes de cas d'utilisation, le chapitre 5 le diagramme de classes, le chapitre 6 les diagrammes de séquence des flux principaux, le chapitre 7 les relations entre entités, le chapitre 8 les bibliothèques utilisées et leur justification, et le chapitre 9 illustre l'application par des captures d'écran annotées.

Chapitre 2

Architecture du système

2.1 Architecture globale — Découplée REST

PORTAL adopte une **architecture découplée** (*decoupled architecture*) : le front-end React communique exclusivement avec le back-end Django via une API REST. Les deux couches sont indépendantes, déployables séparément et testables isolément.

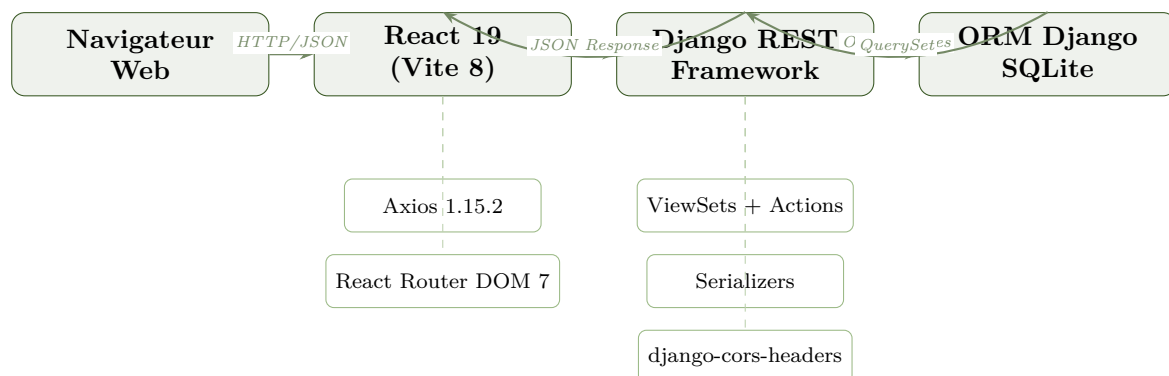


Figure 2.1. Architecture découplée REST de l'application PORTAL

→ **Fonctions associées** : `api.js` (instance Axios centralisée), `App.jsx` (routing + auth state), `ViewSets DRF`, `urls.py`

2.2 Structure des dossiers du projet

Chemin	Contenu
<code>backend/accounts/</code>	Modèles utilisateurs, sérialiseurs, vues, URLs
<code>backend/portal/</code>	Modèles académiques, sérialiseurs, vues, URLs, migrations
<code>backend/emsight_backend/</code>	<code>settings.py</code> , <code>urls.py</code> principal, <code>wsgi/asgi</code>
<code>backend/populate_db.py</code>	Script de peuplement de la base de données
<code>backend/seed_grades.py</code>	Générateur aléatoire de bulletins de notes
<code>frontend/src/components/</code>	<code>TopBar</code> , <code>Sidebar</code> (communs à tous les rôles)
<code>frontend/src/pages/</code>	<code>Login</code> , <code>StudentDashboard</code> , <code>TeacherDashboard</code> , <code>AdminDashboard</code>
<code>frontend/src/api.js</code>	Instance Axios configurée (baseURL)
<code>frontend/src/index.css</code>	Variables CSS, <code>Glassmorphism</code> , thème clair/sombre

Table 2.1. Structure des répertoires du projet PORTAL

2.3 Flux de données — Authentification et routing

Au démarrage, `App.jsx` lit l'utilisateur depuis `localStorage`. S'il n'existe pas, la page `Login.jsx` s'affiche. Après connexion réussie (`POST /api/accounts/users/login/`), l'objet utilisateur (avec ses profils imbriqués) est stocké et le dashboard correspondant au rôle (`StudentDashboard`, `TeacherDashboard` ou `AdminDashboard`) est rendu. L'onglet actif (`activeTab`) est géré dans `App.jsx` et transmis comme prop aux dashboards.

→ **Fonctions associées** : App.jsx : `handleLogin()`, `handleLogout()`, `renderContent()` ; Login.jsx : `handleSubmit()` ; back-end : `UserViewSet.login()`

Chapitre 3

Tables de la base de données

3.1 Application accounts — Utilisateurs et profils

Modèle	Champ	Type / Contrainte
CustomUser	id	BigAutoField (PK)
	username	CharField(150), unique
	email	EmailField
	password	CharField (hashé Django)
	role	CharField {student, teacher, admin}
	matricule	CharField(20), unique, nullable
	gender	CharField {M, F}
	profile_picture	ImageField → media/profile_pics/
StudentProfile	user	OneToOneField → CustomUser (CASCADE)
	filiere	CharField(100) — classe dérivée (ex. 3IIR)
	annee_etude	IntegerField, default=1
	numero_etudiant	CharField(50)
	tutor_name	CharField(150), nullable
TeacherProfile	user	OneToOneField → CustomUser (CASCADE)
	departement	CharField(100)
	matiere	TextField — JSON (liste de modules)
	classes	TextField — JSON (liste de classes)
AdminProfile	user	OneToOneField → CustomUser (CASCADE)
	service	CharField(100)

Table 3.1. Tables de l'application accounts

→ **Fonctions associées** : `UserViewSet` : `login()`, `create_user()`, `assign_modules()`, `change_password()`, `upload_profile_picture()`

3.2 Application portal — Entités académiques

Modèle	Champ	Type / Contrainte
Course	id	BigAutoField (PK)
	name	CharField(200)
	code	CharField(50), nullable
	created_by	FK → CustomUser, SET_NULL
Session	id	BigAutoField (PK)
	course	FK → Course (CASCADE)
	teacher	FK → CustomUser, limit role=teacher, nullable
	target_class	CharField(200), default='All Classes'
	date	DateTimeField
	start_time	TimeField, nullable
	end_time	TimeField, nullable
	created_by	FK → CustomUser, nullable
Attendance	id	BigAutoField (PK)
	session	FK → Session (CASCADE)
	student	FK → CustomUser (CASCADE)
	present	BooleanField, default=False
	<i>Contrainte : unique_together (session, student)</i>	
ReportCard	id	BigAutoField (PK)
	student	FK → CustomUser
	academic_year	CharField(20)
	semester	CharField(20)
	general_average	FloatField, nullable
Grade	id	BigAutoField (PK)
	report_card	FK → ReportCard (CASCADE)
	subject	CharField(100)
	evaluation_type	CharField(50) — CC / Examen / TP
	value	FloatField
	is_rattrapage	BooleanField, default=False
Absence	id	BigAutoField (PK)
	student	FK → CustomUser
	teacher	FK → CustomUser, nullable
	subject	CharField(100)
	date_seance	DateTimeField
	is_present	BooleanField
	justification_text	TextField, nullable
	justification_status	CharField {Pending, Validated, Rejected}
ClassSchedule	id	BigAutoField (PK)
	target_class	CharField(50), unique
	schedule_data	JSONField — liste de 5 listes {time, name}
Notification	id	BigAutoField (PK)
	title	CharField(200)
	content	TextField
	date_envoi	DateTimeField (auto_now_add)
	type_notif	CharField(50)
	sender	FK → CustomUser
	recipients	ManyToManyField → CustomUser
CalendarEvent	id	BigAutoField (PK)
	title	CharField(200)
	start_time	DateTimeField
	end_time	DateTimeField
	event_type	CharField — Cours / TD / Examen
	created_by	FK → CustomUser
	professor	FK → CustomUser, nullable (teacher only)
	target_classes	CharField — filière(s) ou 'All Classes'
DocumentRequest	id	BigAutoField (PK)
	student	FK → CustomUser
	document_type	CharField {Scolarité, Reussite}
	status	CharField {Pending, Validated, Rejected}
	created_at	DateTimeField (auto_now_add)

Table 3.2. Tables de l'application portal

→ **Fonctions associées** : Chaque table est gérée par son ViewSet DRF correspondant : `CourseViewSet`, `SessionViewSet`, `AttendanceViewSet`, `GradeViewSet`, `ReportCardViewSet`, `AbsenceViewSet`, `ClassScheduleViewSet`, `NotificationViewSet`, `CalendarEventViewSet`, `DocumentRequestViewSet`

3.3 Historique des migrations

Fichier	Date	Contenu
accounts/0001_initial.py	2026-04-25	CustomUser, StudentProfile, TeacherProfile, AdminProfile
accounts/0002_customuser_matricule.py	2026-04-26	Ajout matricule
accounts/0003_customuser_gender.py	2026-04-26	Ajout gender
accounts/0004_customuser_profile_picture.py	2026-04-28	Ajout profile_picture
accounts/0005_alter_teacherprofile_matiere.py	2026-05-06	TextField pour matiere
accounts/0006_teacherprofile_classes.py	2026-05-06	Ajout classes
portal/0001_initial.py	2026-04-25	CalendarEvent, Notification, ReportCard, Grade, Absence, DocumentRequest
portal/0002_..._professor_target.py	2026-05-03	professor + target_classes dans CalendarEvent
portal/0003_classschedule.py	2026-05-07	Création ClassSchedule
portal/0004_course_session_attendance.py	2026-05-10	Création Course, Session, Attendance

Table 3.3. Historique des migrations Django

Chapitre 4

Diagrammes de cas d'utilisation

4.1 Vue globale du système

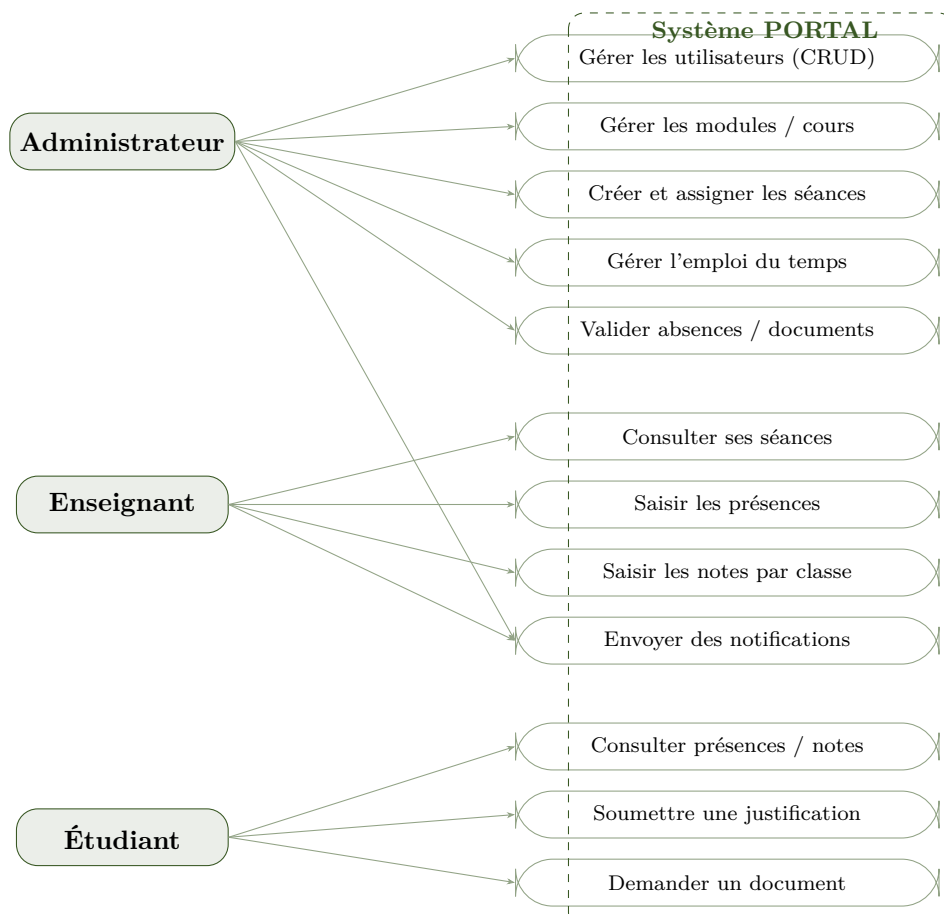


Figure 4.1. Diagramme de cas d'utilisation global — PORTAL

→ **Fonctions associées** : Toutes les actions passent par l'API REST ; la redirection vers le bon dashboard est gérée par `App.jsx` → `renderContent()` selon `user.role`

4.2 Cas d'utilisation détaillé : Saisie des présences

Acteur principal : Enseignant **Précondition** : Authentifié, séances existantes **Postcondition** : Présences enregistrées en base (upsert)

Scénario nominal :

1. L'enseignant ouvre l'onglet « Mark Presence » dans son dashboard.
2. Il crée une nouvelle séance (cours + classe + date + horaires) via le formulaire.
3. Il sélectionne une séance dans la liste déroulante — les présences existantes sont chargées

(GET /api/portal/attendances/?session=id).

4. Pour chaque étudiant de la classe, il bascule le sélecteur Présent/Absent.
5. Il clique « Save Attendance » : un appel POST /api/portal/attendances/bulk_create/ est envoyé.
6. Le back-end effectue un `update_or_create` pour chaque ligne.
7. Un message de succès s'affiche et la liste est rechargée.

→ **Fonctions associées** : TeacherDashboard : `handleCreateSession()`, `handleSelectSession()`, `toggleAttendance()`, `saveAttendance()` ; Back-end : `AttendanceViewSet.bulk_create()`

4.3 Cas d'utilisation détaillé : Workflow justification d'absence

Acteurs : Étudiant (initiateur), Administrateur (valideur)

Scénario :

1. L'étudiant consulte ses absences dans l'onglet « Absences ».
2. Pour une absence sans justification, il clique « Justify » et saisit un texte.
3. Soumission : PATCH /api/portal/absences/{id}/ avec `justification_text` et `justification_status=Pending`.
4. L'administrateur voit la justification dans la « Approval Queue ».
5. Il clique « Approve » ou « Reject » : PATCH avec le nouveau `justification_status`.
6. L'étudiant voit le badge mis à jour (Validated / Rejected) à son prochain chargement.

→ **Fonctions associées** : StudentDashboard : `handleJustifyAbsence()` ; AdminDashboard : `handleValidation('absence', ...)` ; Back-end : `AbsenceViewSet`

Chapitre 5

Diagramme de classes

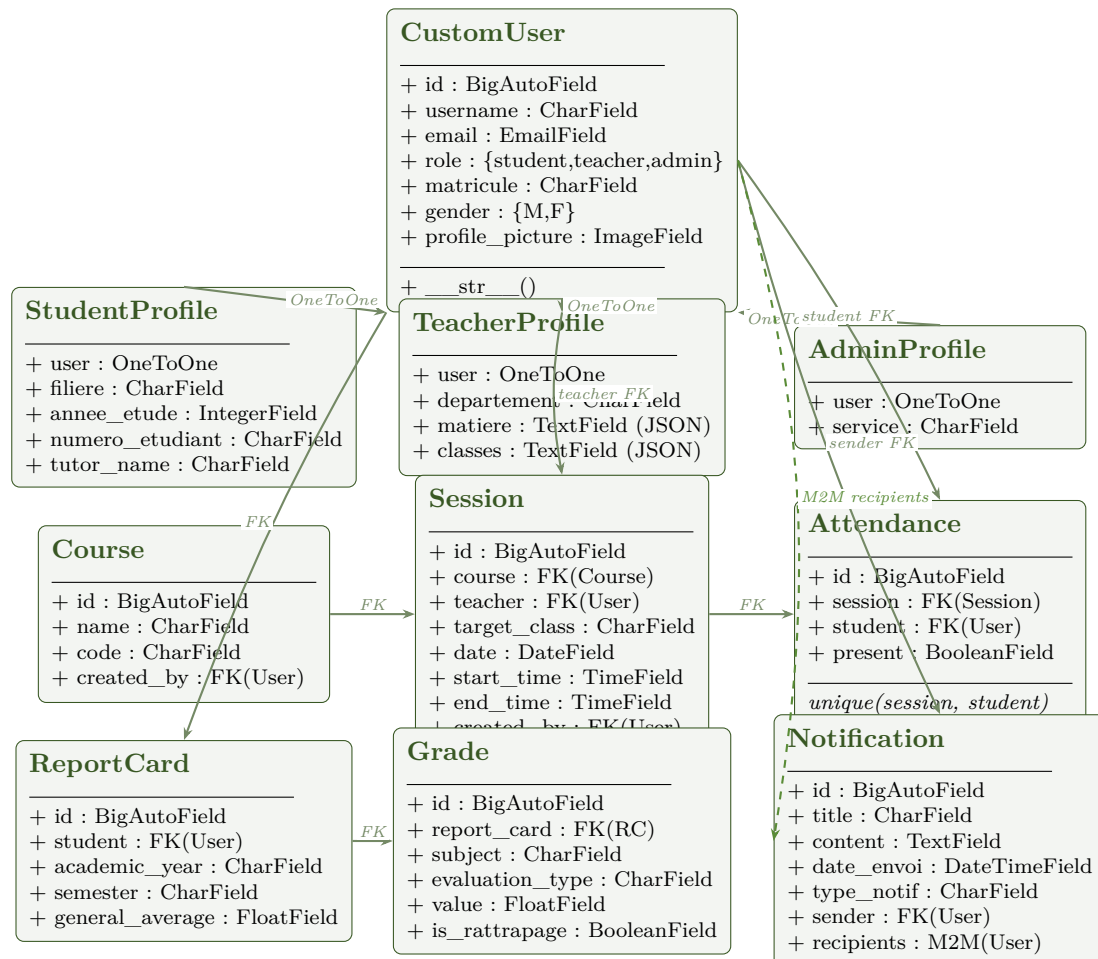


Figure 5.1. Diagramme de classes simplifié — modèles principaux de PORTAL

Note : Les modèles `Absence`, `ClassSchedule`, `CalendarEvent` et `DocumentRequest` ont tous une clé étrangère vers `CustomUser` (student / created_by) et suivent le même patron ; ils ont été omis pour préserver la lisibilité du diagramme.

→ **Fonctions associées :** Chaque classe Django correspond à un `ModelViewSet` DRF avec un sérialiseur dédié (`portal/serializers.py`)

Chapitre 6

Diagrammes de séquence

6.1 Séquence 1 : Connexion utilisateur

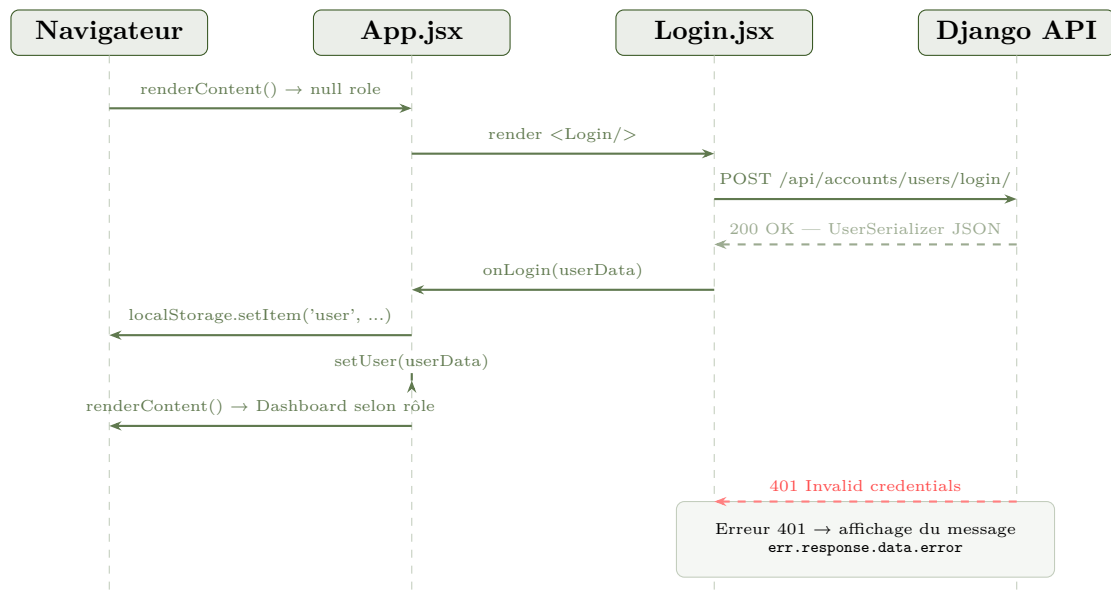


Figure 6.1. Diagramme de séquence — Connexion utilisateur

→ **Fonctions associées** : Login.jsx : `handleSubmit()`; App.jsx : `handleLogin()`; Back-end : `ViewSet.login()`

6.2 Séquence 2 : Saisie des présences (Enseignant)

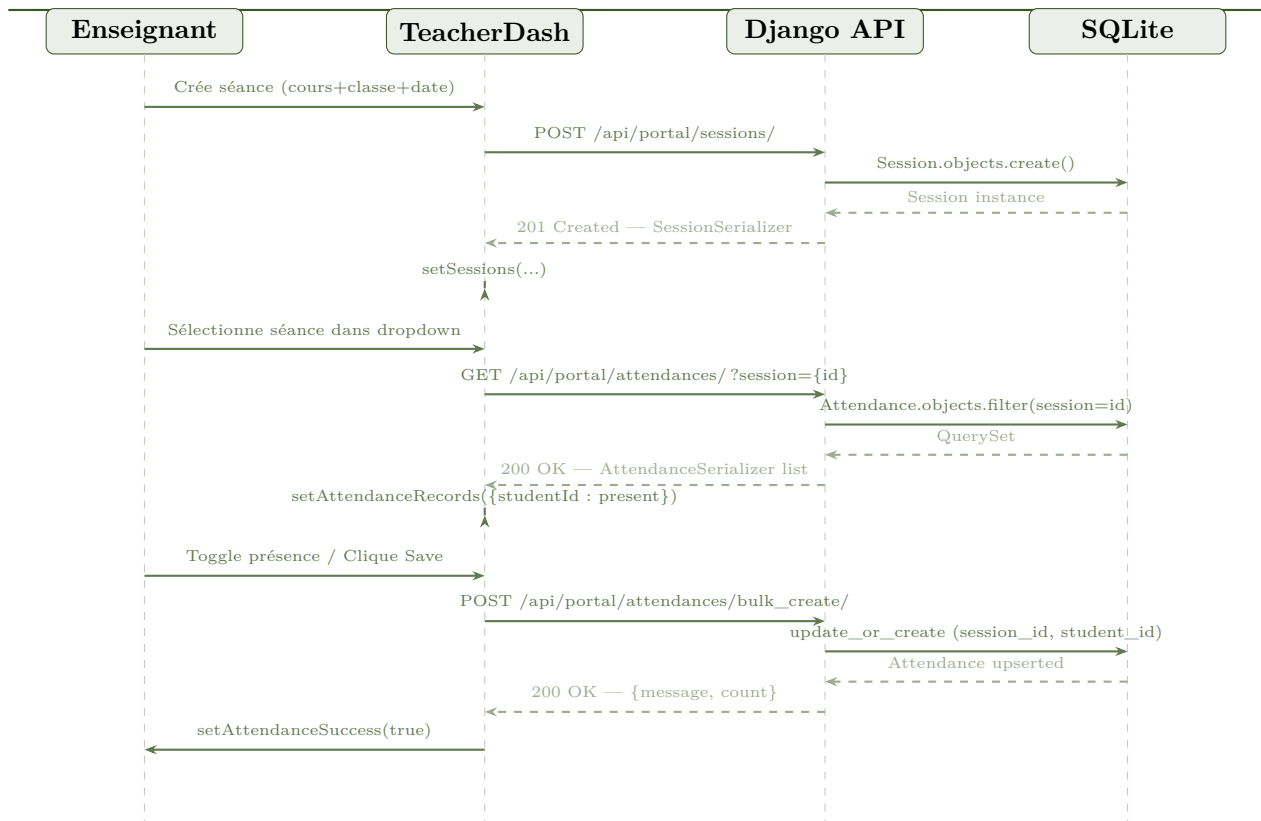


Figure 6.2. Diagramme de séquence — Saisie des présences par un enseignant

→ **Fonctions associées :** TeacherDashboard : `handleCreateSession()`, `handleSelectSession()`, `toggleAttendance()`, `saveAttendance()`; Back-end : `SessionViewSet.create()`, `AttendanceViewSet.bulk_create()`

6.3 Séquence 3 : Saisie groupée des notes

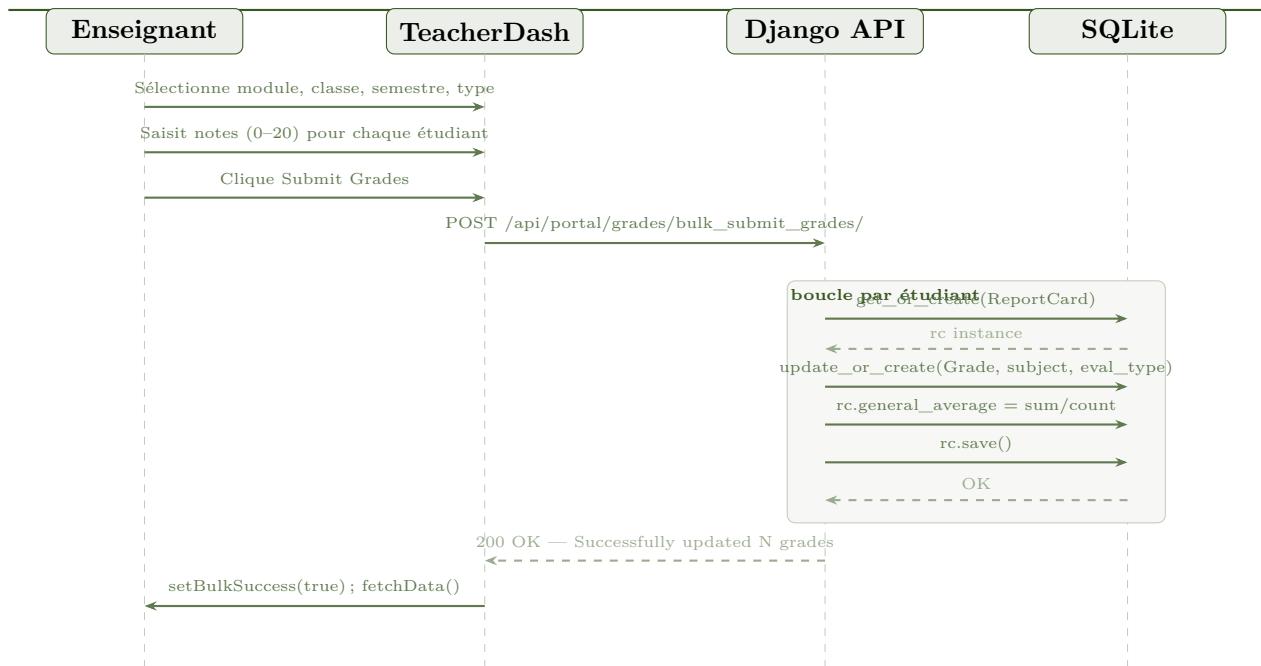


Figure 6.3. Diagramme de séquence — Saisie groupée des notes par un enseignant

→ **Fonctions associées :** TeacherDashboard : submitClassGrades(); Back-end : GradeViewSet.bulk_submit_grades()

Chapitre 7

Relations entre les entités

7.1 Carte des relations

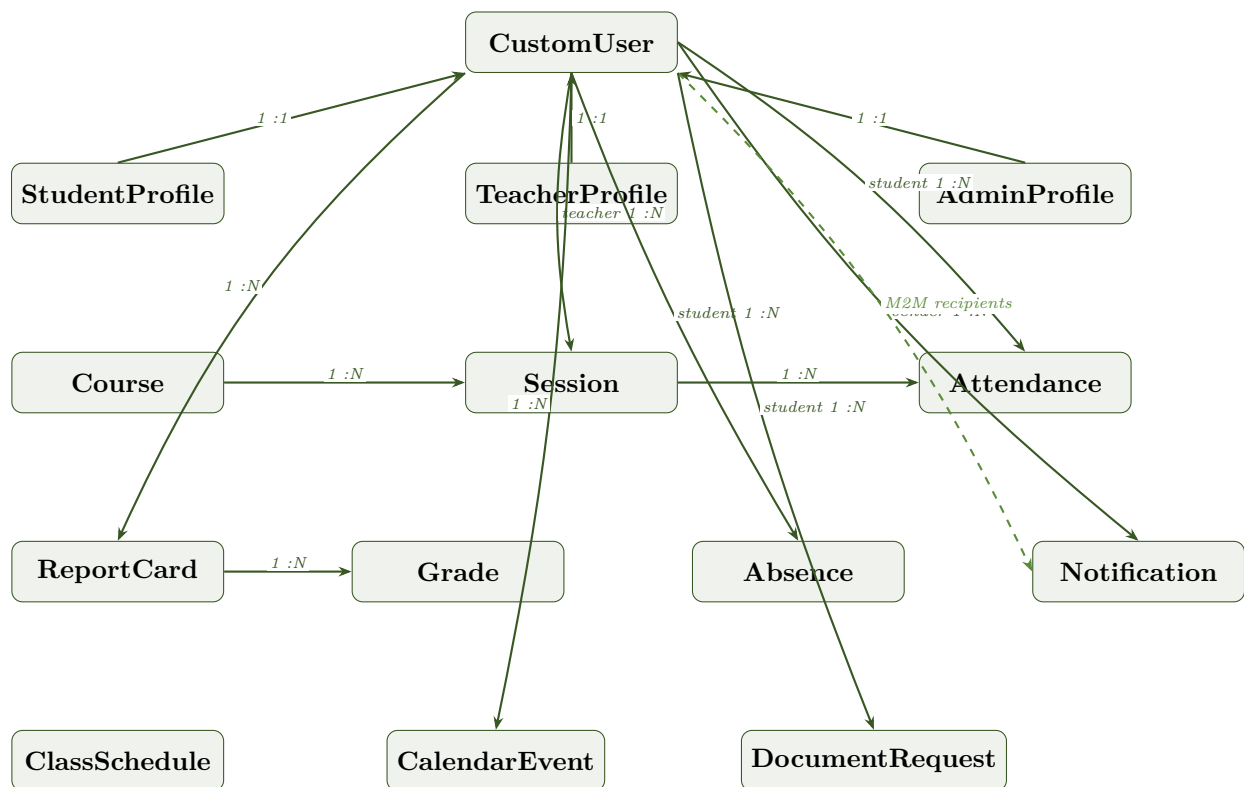


Figure 7.1. Carte complète des relations entre entités — PORTAL

7.2 Description détaillée des relations

Entité source	Cardinalité	Entité cible	Sémantique et comportement
CustomUser	1 :1	StudentProfile	Profil étendu étudiant — CASCADE à la suppression
CustomUser	1 :1	TeacherProfile	Profil étendu enseignant — CASCADE
CustomUser	1 :1	AdminProfile	Profil étendu administrateur — CASCADE
Course	1 :N	Session	Un cours possède plusieurs séances — CASCADE
CustomUser (teacher)	1 :N	Session	Un enseignant anime plusieurs séances — SET_NULL
Session	1 :N	Attendance	Une séance génère une liste de présences — CASCADE
CustomUser (student)	1 :N	Attendance	Un étudiant a des enregistrements de présence — CASCADE
<i>unique_together</i>	—	(Session, Student)	Garantit une seule ligne par étudiant/séance — contrainte d'unicité
CustomUser (student)	1 :N	ReportCard	Un étudiant a plusieurs bulletins — CASCADE
ReportCard	1 :N	Grade	Un bulletin contient plusieurs notes — CASCADE
CustomUser (student)	1 :N	Absence	Un étudiant cumule des absences — CASCADE
CustomUser (sender)	1 :N	Notification	Un expéditeur crée plusieurs notifs — CASCADE
CustomUser	M :N	Notification.recipients	Plusieurs utilisateurs reçoivent une notif — ManyToMany
CustomUser	1 :N	CalendarEvent	Un utilisateur crée des événements — CASCADE
CustomUser (student)	1 :N	DocumentRequest	Un étudiant fait des demandes — CASCADE
ClassSchedule	—	(aucune FK sortante)	Singleton par classe (unique target_class); JSONField auto-suffisant

Table 7.1. Description détaillée des relations entre entités

7.3 Sérialeurs imbriqués — Relations exposées à l'API

Les sérialiseurs DRF matérialisent les relations sous forme de JSON imbriqué afin de limiter le nombre de requêtes côté front-end.

Sérialiseur	Données imbriquées exposées
UserSerializer	student_profile, teacher_profile, admin_profile (read_only)
ReportCardSerializer	grades (GradeSerializer, many=True, read_only) + student_details (UserSerializer)
AttendanceSerializer	session_details (SessionSerializer) + student_details (UserSerializer)
SessionSerializer	course_name, teacher_name, created_by_name (SerializerMethodField)
CalendarEventSerializer	created_by_name, professor_name (SerializerMethodField)
NotificationSerializer	sender_name, sender_role (SerializerMethodField)
AbsenceSerializer	student_details (UserSerializer)
DocumentRequestSerializer	student_details (UserSerializer)

Table 7.2. Sérialeurs DRF et données relationnelles imbriquées

→ **Fonctions associées** : Tous les sérialiseurs sont définis dans `portal/serializers.py` et `accounts/serializers.py`

Chapitre 8

Bibliothèques utilisées et justification

8.1 Stack back-end (Python/Django)

Bibliothèque	Version	Rôle et justification
Django	6.0.4	Framework web Python principal. Fournit l'ORM (mapping objet-relationnel vers SQLite), le système de migrations, l'administration auto-générée, l'authentification native (hashage des mots de passe, <code>authenticate()</code> , <code>AbstractUser</code>), et le routing URL. Choisi pour sa maturité, sa documentation exhaustive et l'adéquation avec le CDC.
djangorestframework	3.17.1	Transforme Django en serveur d'API REST. Fournit <code>ModelViewSet</code> (CRUD auto), <code>@action</code> (endpoints personnalisés <code>bulk_create</code> , <code>bulk_submit_grades</code> , <code>login...</code>), les sérialiseurs imbriqués, et la navigation API navigable. Indispensable pour l'architecture découplée.
django-cors-headers	4.9.0	Gestion des en-têtes CORS (<i>Cross-Origin Resource Sharing</i>). Indispensable car le front-end React (port 5173) et le back-end Django (port 8000) tournent sur des origines différentes en développement. La configuration <code>CORS_ALLOW_ALL_ORIGINS = True</code> simplifie le développement local.
Pillow	12.2.0	Bibliothèque de traitement d'images Python, requise par le champ <code>ImageField</code> de Django pour la gestion des photos de profil (stockage dans <code>media/profile_pics/</code>). Sans Pillow, Django refuse le démarrage dès qu'un <code>ImageField</code> est déclaré.
python-dotenv	1.2.2	Chargement des variables d'environnement depuis un fichier <code>.env</code> . Permet de stocker les clés secrètes (ex. <code>GEMINI_API_KEY</code>) hors du code source versionné, conformément aux bonnes pratiques de sécurité (12-Factor App).
SQLite (intégré Django)	—	Base de données relationnelle légère, sans serveur dédié, intégrée à Python. Parfaitement adaptée au contexte d'un mini-projet académique : installation zéro, fichier unique (<code>db.sqlite3</code>), support complet des relations SQL (FK, <code>unique_together</code> , <code>ManyToMany</code>).

Table 8.1. Bibliothèques back-end — rôle et justification

8.2 Stack front-end (JavaScript/React)

Bibliothèque	Version	Rôle et justification
React	19.2.5	Bibliothèque UI déclarative de Meta. Le modèle de composants (<code>useState</code> , <code>useEffect</code> , <code>props</code>) permet de construire les trois dashboards comme des arbres de composants réutilisables. La version 19 apporte des optimisations du rendu (<i>concurrent features</i>).
Vite	8.0.10	Outil de build et serveur de développement ultra-rapide basé sur Rolldown/ESM natif. Fournit le Hot Module Replacement (HMR), ce qui accélère considérablement le cycle développement : la modification d'un composant se reflète instantanément dans le navigateur sans recharge complète.
Axios	1.15.2	Client HTTP basé sur les Promises, plus ergonomique que <code>fetch</code> natif. Il offre la sérialisation JSON automatique, l'intercepteur d'erreurs (<code>err.response.data</code>), et la configuration centralisée de la <code>baseUrl</code> dans <code>api.js</code> . Utilisé pour tous les appels REST vers Django.
React Router DOM	7.14.2	Routing côté client pour les SPA. Bien que la navigation dans PORTAL soit principalement gérée par <code>activeTab</code> (state React), React Router DOM est utilisé pour la structure d'URL et les navigations futures (pages dédiées par rôle).
Lucide React	1.11.0	Bibliothèque d'icônes SVG légères et cohérentes (500+ icônes). Utilisée dans la Sidebar (<code>LayoutDashboard</code> , <code>Calendar</code> , <code>Bell</code> , <code>UserCheck...</code>), la TopBar (<code>Logout</code> , <code>Settings</code> , <code>Moon/Sun</code>) et la page Login (<code>Mail</code> , <code>Lock</code> , <code>ArrowRight</code>). Remplace avantageusement les polices d'icônes (pas de FOUT, tree-shakable).
i18next + react-i18next	26.0.9 17.0.6	/ Framework d'internationalisation. Prépare l'application à supporter plusieurs langues (français, anglais, arabe) sans refactoring majeur. La détection de langue du navigateur est assurée par <code>i18next-browser-languagedetector</code> .

Table 8.2. Bibliothèques front-end — rôle et justification

8.3 Dépendances de développement

Package	Version	Usage
<code>@vitejs/plugin-react</code>	6.0.1	Plugin Vite pour le support JSX/React (Babel transform)
<code>eslint + plugins</code>	10.2.1	Linter JavaScript — détection d'erreurs et mauvaises pratiques
<code>eslint-plugin-react-hooks</code>	7.1.1	Règles lint spécifiques aux React Hooks
<code>@types/react</code>	19.2.14	Types TypeScript pour l'autocomplétion IDE

Table 8.3. Dépendances de développement (devDependencies)

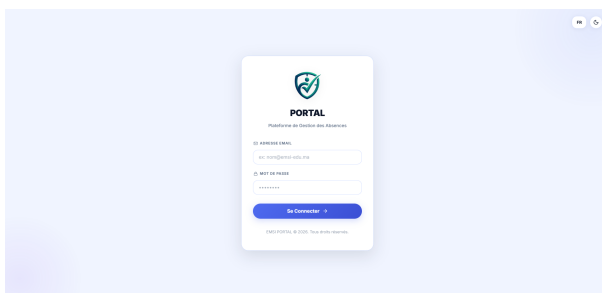
Chapitre 9

Captures d'écran

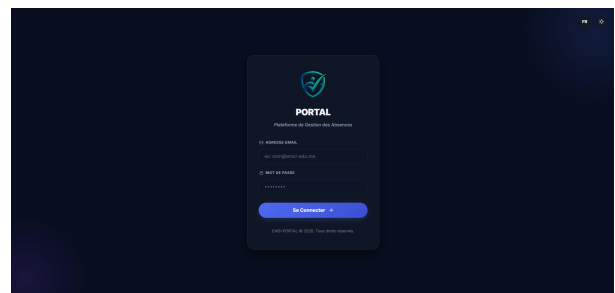
Remarque

Chaque capture est accompagnée d'annotations indiquant les fonctions React et les endpoints API mis en jeu. Insérer les fichiers `.png` correspondants dans Overleaf.

9.1 Authentification



(a) Page de connexion — formulaire email/mot de passe, thème clair, logo PORTAL



(b) Page de connexion — thème sombre (bascule Moon/-Sun)

Figure 9.1. Page de connexion (Login.jsx)

→ **Fonctions associées** : Login.jsx : `handleSubmit()` → `POST /api/accounts/users/login/`; gestion erreur 401; `setTheme()` pour la bascule thème

9.2 Tableau de bord Administrateur

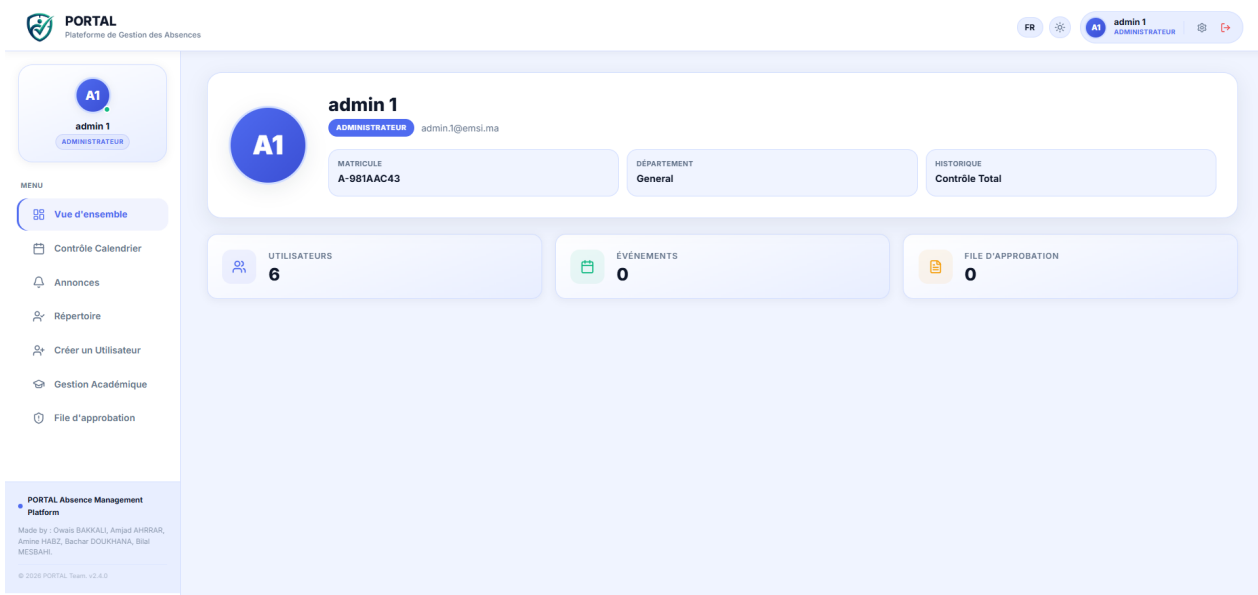
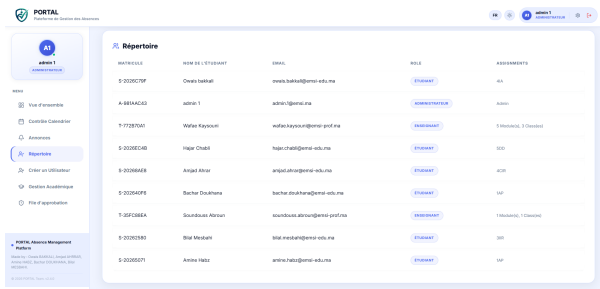
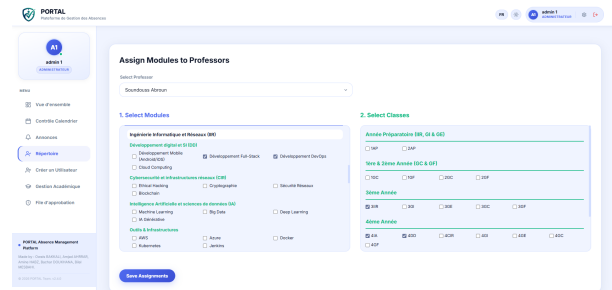


Figure 9.2. Vue d'ensemble Administrateur — statistiques (utilisateurs, documents en attente, absences)

→ **Fonctions associées** : AdminDashboard.jsx : renderOverview(); données chargées par fetchData() via GET /api/accounts/users/, GET /api/portal/absences/, GET /api/portal/document-requests/



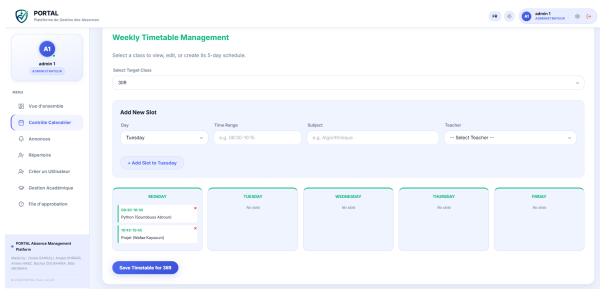
(a) Annuaire — création utilisateur avec dérivation automatique de classe



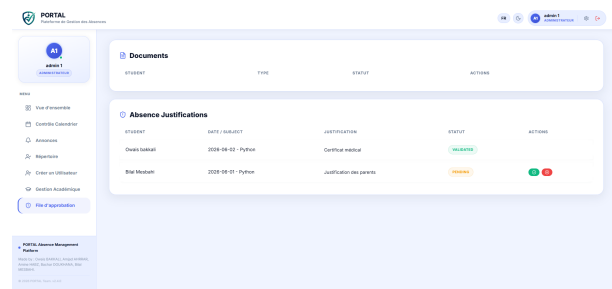
(b) Affectation de modules et classes à un enseignant

Figure 9.3. Gestion des utilisateurs et affectation académique (AdminDashboard.jsx — renderUsers())

→ **Fonctions associées** : handleCreateUser() → POST /api/accounts/users/create_user/;
handleSaveModules() → PATCH /api/accounts/users/{id}/assign_modules/;
deriveClass(secteur, grade, subSector)



(a) Éditeur d'emploi du temps par classe — ajout et suppression de créneaux

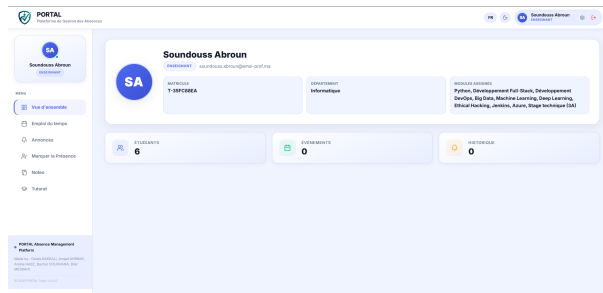


(b) File de validation — absences et demandes de documents

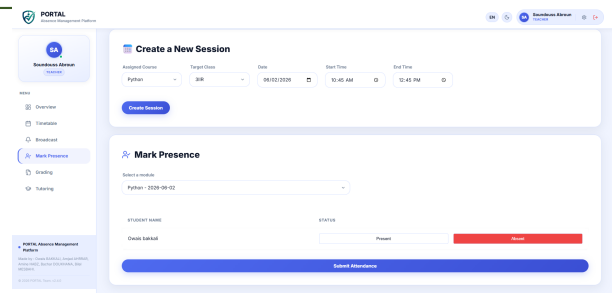
Figure 9.4. Emploi du temps et validations (AdminDashboard.jsx)

→ **Fonctions associées** : Emploi du temps : handleAddSlot(), handleRemoveSlot(), handleSaveTimetable() → POST/PATCH /api/portal/schedules/; Validations : handleValidation(type, id, status) → PATCH /api/portal/absences/{id}/ ou PATCH /api/portal/document-requests/{id}/

9.3 Tableau de bord Enseignant



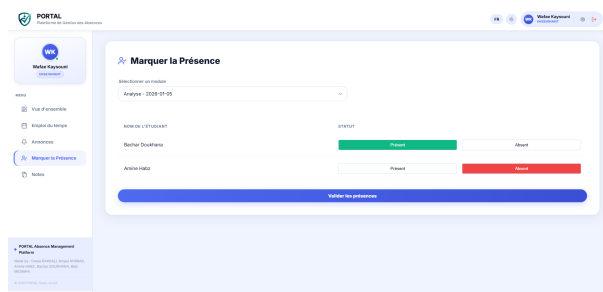
(a) Vue d'ensemble enseignant — profil, stats, modules assignés



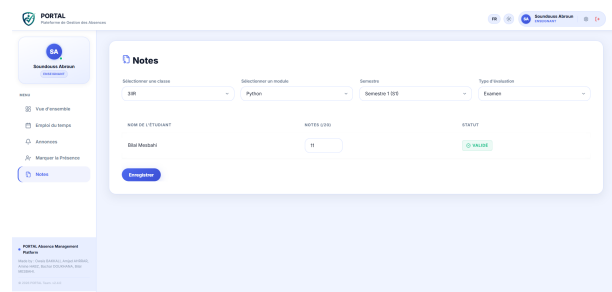
(b) Création d'une séance — cours + classe + date + horaires

Figure 9.5. Dashboard et création de séance (TeacherDashboard.jsx)

→ **Fonctions associées** : Profil : `renderOverview()`; `teacherModules` et `teacherClasses` parsés depuis le JSON du `TeacherProfile`; `handleCreateSession()` → POST `/api/portal/sessions/`



(a) Saisie des présences — sélecteur Présent/Absent par étudiant

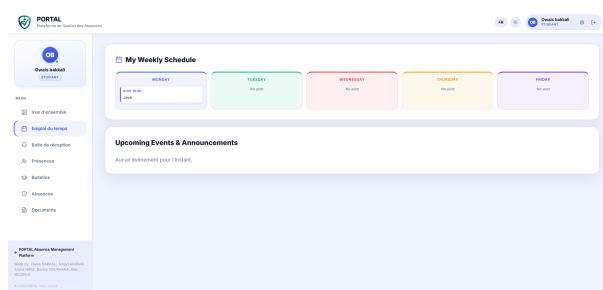


(b) Saisie groupée des notes — filtres module / semestre / type d'évaluation

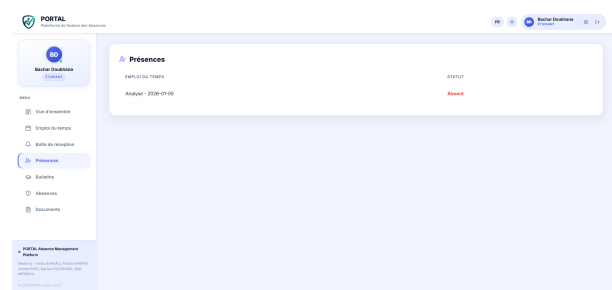
Figure 9.6. Présences et notes (TeacherDashboard.jsx — `renderAttendance()`, `renderGrades()`)

→ **Fonctions associées** : Présences : `handleSelectSession()`, `toggleAttendance()`, `saveAttendance()` → POST `/api/portal/attendances/bulk_create/`;
Notes : `handleClassGradeChange()`, `submitClassGrades()` → POST `/api/portal/grades/bulk_submit_grades/`

9.4 Tableau de bord Étudiant



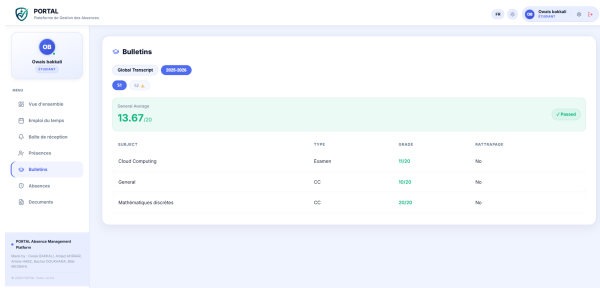
(a) Emploi du temps hebdomadaire — 5 colonnes colorées par jour



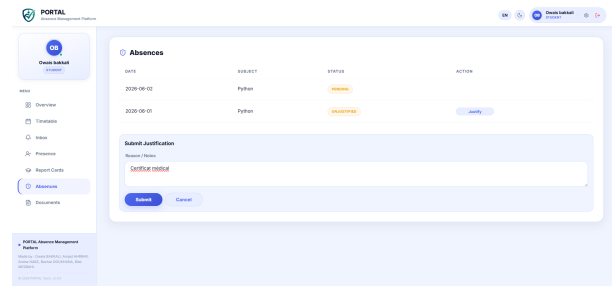
(b) Suivi des présences — tableau cours/classe/date/statut

Figure 9.7. Emploi du temps et présences (StudentDashboard.jsx — `renderCalendar()`, `renderAttendance()`)

→ **Fonctions associées** : `renderCalendar()` : GET `/api/portal/schedules/?target_class={filiere}`; `renderAttendance()` : GET `/api/portal/attendances/?student={id}`



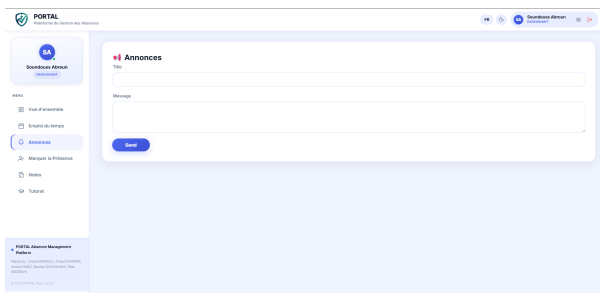
(a) Relevé global multi-années — onglets par année + semestre



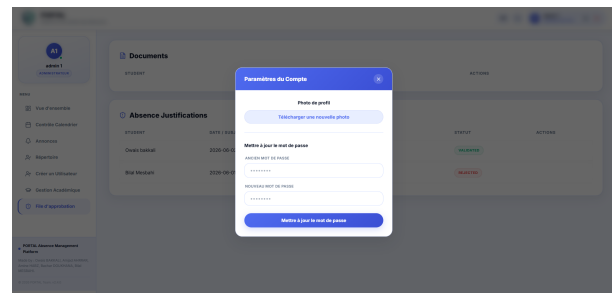
(b) Gestion des absences — formulaire de justification

Figure 9.8. Notes et absences (StudentDashboard.jsx — `renderGrades()`, `renderAbsences()`)

→ **Fonctions associées** : `renderGrades()` : GET `/api/portal/report-cards/` + filtrage par `academic_year` + `semester` + vue « Relevé global »; `handleJustifyAbsence()` → PATCH `/api/portal/absences/{id}/`



(a) Notifications — suppression unitaire (icône poubelle) et globale



(b) Modal Paramètres — mot de passe + upload photo de profil

Figure 9.9. Notifications et paramètres (StudentDashboard.jsx + TopBar.jsx)

→ **Fonctions associées** : Notifications : `handleDeleteNotification(id)` → DELETE `/api/portal/notifications/{id}/`; `handleClearAllNotifications()` (optimistic UI); Paramètres : `handleChangePassword()` + `handleProfilePicUpload()` (FormData multipart)

Conclusion

Le projet **PORTAL** répond intégralement aux spécifications du Projet 5 du cahier des charges tout en les dépassant largement. Les fonctionnalités requises — authentification, gestion des étudiants, des modules, des enseignants, des séances et saisie des présences — sont toutes implémentées et démontrables. Les extensions (notes, bulletins, emploi du temps, notifications, justifications, documents) illustrent la capacité de l'équipe à livrer un produit complet et à forte valeur ajoutée.

Ce rapport a présenté l'intégralité du projet selon une progression logique : introduction contextuelle, architecture découplée REST, description des tables de la base de données, diagrammes UML (cas d'utilisation, classes, séquences), carte des relations inter-entités, justification technique de chaque bibliothèque, et captures d'écran annotées avec les fonctions associées.

Compétences consolidées :

- Architecture REST découplée avec DRF et React
- Modélisation relationnelle avancée (unique_together, M2M, JSONField)
- Sérialeurs imbriqués et actions personnalisées (bulk_create, bulk_submit_grades)
- Gestion d'état, rendu conditionnel et cycle de vie des composants React
- Collaboration en équipe avec Git et répartition des tâches

Perspectives : JWT pour l'authentification sans état (`django-rest-framework-simplejwt`); notifications push e-mail/SMS (`Celery` + `Redis`); génération de bulletins PDF automatisés (`WeasyPrint`); application mobile de pointage par QR Code (React Native); déploiement cloud (Railway, Render, ou VPS Ubuntu).